Merlino2D User Manual

Fabio Ragazzi

Contents

1 Introduction

Merlino2D is an open-source plasma simulation code designed to provide a fast and user-friendly platform for modeling a broad range of plasma devices and gas discharges. The code is based on a drift–diffusion fluid framework and can accommodate detailed kinetic reaction schemes, depending on the chosen mesh resolution. Merlino2D employs a fully implicit time-integration scheme with adaptive time stepping, enabling stable and computationally efficient simulations. The code is implemented in MATLAB and supports two-dimensional plasma simulations on unstructured triangular meshes generated using Gmsh.

2 Installation

2.1 Download

Here you can download a ZIP folder containing the code. Alternatively, you can clone the repository with git:

```
> git clone https://github.com/PTL-Unibo/Merlino2D.git
```

2.2 Installation Steps

To use Merlino2D you need MATLAB installed on your computer. You also need to install Gmsh for mesh generation.

The first time using Merlino2D it is necessary to run the script **Merlino2D_startup.m** that is inside the **src/** folder. By running this script, you will be firstly asked to select the **gmsh.exe** executable that you should have previously installed on your computer. After, you will be asked to select the **Code/** folder of the LoKI-B repository, that you should have cloned on your device if you wish to have the option of using the Boltzmann solver for computing swarm parameters and rate coefficients.

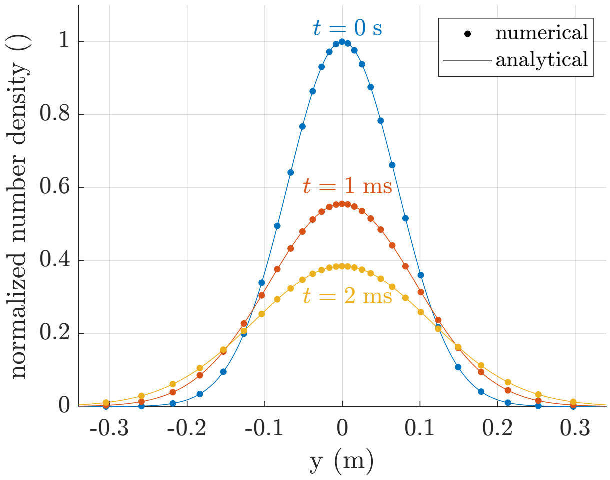
If the folder containing the project is moved to another location, you need to run **Merlino2D_startup.m** again.

2.3 Verification

If the installation was successful, you can now run all the examples contained in the **cases/** folder (if you also moved the **Air.in** file inside the **Code/Input/** folder of the LoKI-B repository). We suggest that you start by running the script **c_Diffusion.m**, which should not take a long time and does not require to move the **Air.in** file. By running that script, a Gmsh window containing a square divided into two half should appear. Closing that window the simulation will start and you should see the following text appearing in the Command Window:

```
Generated Mesh
Photoionization is OFF
Initialization finished
Updated photoionization, maximum value = 0.000000e+00
Simulation finished
Deleted SquareCenterRefined.m
```

Finally, the following figure should appear:



If you see the above figure appearing without getting any errors, it means that you have successfully installed Merlino2D.

3 Basic Workflow

Every time you open MATLAB, to use the code you have to run the script **init.m** that is inside the main folder. You can do it by selecting the file and pressing **F9**, or typing in the Command Window

```
>> init
```

This will add to the MATLAB path the folder **src/** that contains the code.

The core of the code is the function `Merlino2D`. The syntax of the function is:

```
out = Merlino2D(opts,"key1",value1,"key2",value2,...);
```

`opts` is a structure that contains the input parameters for the simulation. Some parameters have a default value that can be found inside **DefaultMerlino2Dinput.m**.
The key-value arguments are optional and can be used to overwrite the parameters passed through `opts`.

`out` is the unprocessed output structure that is given in input to the `PostProcessing` function to obtain the post-processed output structure `out_pp`.

```
out_pp = PostProcessing(out);
```

`out_pp` can be used to plot the results using the `Plot` function.

```
Plot(out_pp,"type",type_value);
```

To save the results of a simulation, use the `Save` function.

```
Save(out_pp,"my_result.mat");
```

To retrieve the results of a simulation that was previously saved, use the `Load` function.

```
out_pp = Load("my_result.mat");
```

It is also possible to export the results for visualization with Paraview using the `ExportVTU` function.

```
ExportVTU(out_pp,"my_results")
```

3.1 Post Processing

There are two types of post processing, a lighter version and a more complete one. To select which version to use, you can give to the function the string `"light"` or `"full"` as the second argument (`"light"` is the default).

```
out_pp = PostProcessing(out,"light");
out_pp = PostProcessing(out,"full");
```

The lighter version will generate an output structure that contains:

- the simulation time instants
- the number density of all the species at cells
- the surface charge density at the dielectric interface (if a dielectric material exists inside the domain)
- the electric potential at nodes
- the electric field at cells
- the charge density at cells
- the current over time computed with Sato's equation
- the applied voltage over time
- information about chemical species
- information about the mesh

The full version adds to the previous list:

- the electric field at faces
- the electric field at nodes
- the number density of all the species at nodes
- the charge density at nodes
- the flux density at faces
- the chemistry source term
- the current over time computed integrating the current density at electrodes

3.2 Plot

The `Plot` function can plot several output quantities and will return the axes object `ax`, containing the plot. It is possible to specify which type of plot to generate with the key-value argument `"type"`.

```
ax = Plot(out_pp,"type",<type_value>);
```

`<type_value>` needs to be a string. The available options are:

- `"nc"`, to plot the species number density at cells

- `"nn"`, to plot the species number density at nodes
- `"rhoc"`, to plot the charge density at cells
- `"rhon"`, to plot the charge density at nodes
- `"phi"`, to plot the electric potential at nodes
- `"v-i"`, to plot the v-i characteristic
- `"Ec"`, to plot the electric field at cells
- `"|Ec|"`, to plot the electric field module at cells
- `"|En|"`, to plot the electric field module at nodes
- `"t-iv"`, to plot the current and voltage over time (in a single plot)
- `"t-Fx"`, to plot the force along x over time
- `"fxc"`, to plot the force density along x at cells
- `"fxc_log"`, to plot the force density along x at cells in log scale
- `"fxn"`, to plot the force density along x at nodes
- `"fxn_log"`, to plot the force density along x at nodes in log scale
- `"rhon_log"`, to plot the charge density at nodes in log scale
- `"sigma"`, to plot the surface charge density
- `"sph"`, to plot the photoionization source term
- `"msh"`, to plot only the used mesh

It is possible to pass to the `Plot` function other key-value arguments. The available keys are:

- `"ax"`: allows to pass the desired axes object to display the plot
- `"species_index`: index of the chemical species to plot
- `"k"`: index of the time instant to display
- `"flip_y"`: if equal to 1, plots also the symmetry of the domain with respect to the x-axis (for 2D color plots) and also **doubles the current**.
- `"log10_zero_val"`: for log scale plots sets the threshold under which everything is considered to be zero

3.3 Save

The `Save` function needs a post processed output structure as input. Only the fields contained in the structure will be saved in a **.mat** file, so, be careful to the type of post processing that was performed (`"light"` or `"full"`). The second input argument is an optional string that specifies the file where to save the results. If no second argument is given, the results will be saved with a string indicating the moment when the function was called, like: `"2025_12_25_12_30_00"`.

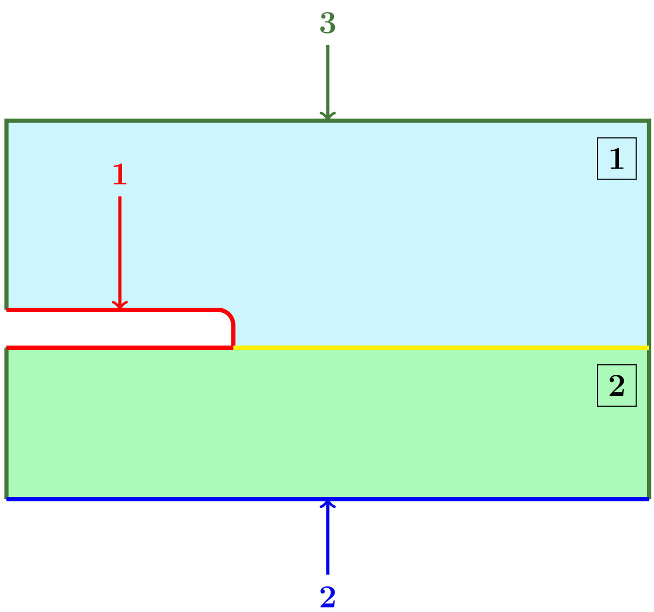
```
Save(out_pp,"my_result.mat");
Save(out_pp);
```

3.4 Load

The `Load` function takes as argument a string specifying the **.mat** file containing the results you want to load and returns the post processed output structure.

```
out_pp = Load("my_result.mat");
```

4 Numerical Identifiers of the Mesh



Let us consider an example domain composed of a gas region (above the yellow line) and a solid dielectric region (below the yellow line), separated by a dielectric interface (the yellow line). Suppose that the user intends to apply three different boundary conditions to the regions highlighted in red, blue, and green.

When generating a mesh to use in Merlino2D, each external boundary where a distinct boundary condition is required must be assigned a unique numerical identifier. For instance, in this case, the three external regions can be labeled with the identifiers 1, 2, and 3, respectively. These identifiers are defined in the **.geo** file using the `Physical Curve` command. Similarly, each subdomain corresponding to a different relative permittivity value ε_r must be assigned a unique identifier. In the present example, the gas and solid dielectric regions are labeled as 1 and 2, respectively. These identifiers are defined in the **.geo** file using the `Physical Surface` command.

Note that **Merlino2D treats as gas only regions with a surface identifier equal to 1**, whereas all regions assigned a numerical identifier greater than 1 are interpreted as solid dielectric materials. Note also that **Merlino2D identifies as dielectric interfaces only lines assigned a numerical identifier greater than or equal to 1000**. For this reason, in the **.geo** file, the yellow line can be labeled, for example, with the identifier 1005 using the `Physical Curve` command.

In general, a mesh provided as input to Merlino2D includes identifiers for the external boundaries, hereafter referred to as **BoundaryIDs** (1, 2, and 3 in the previous example), as well as identifiers for the subdomains, hereafter referred to as **SurfaceIDs** (1 and 2 in the previous example).

5 Species Database

All species used in a simulation—whether defined within the `reactions` variable of the kinetic scheme specified by `CHEMICAL_MODEL`, or listed in the `SPECIES_NO_CHEM` input parameter—must be included in the **species_database.csv** file located in the **data/** folder. This file is a comma-separated values (CSV) file containing three columns.

- The first column lists the names of the species. Every species name used in the `reactions` variable or appearing in `SPECIES_NO_CHEM` must be present in this first column of **species_database.csv**.
- The second column contains the mass of the species, expressed in kg.
- The third column contains the charge of the species.

```
NAME,       MASS,                     CHARGE
e,          0.000000910938370e-24,    -1
N2,         0.046496180670873e-24,    0
O2,         0.053138492195284e-24,    0
N,          0.023247179397067e-24,    0
O,          0.026569246097642e-24,    0
N2+,        0.046495269732503e-24,    +1
O2+,        0.053137581256914e-24,    +1
O4+,        0.106276073452198e-24,    +1
O-,         0.026569246097642e-24,    -1
O2-,        0.053139403133654e-24,    -1
O3-,        0.079707738292926e-14,    -1
I+,         0.049816425494708e-24,    +1
I-,         0.053139403133654e-24,    -1
```

New species that the user wants to consider in a simulation must be added here.

6 Input Parameters

6.1 MSH

This parameter is a string used to specify the mesh file. For example, if you have created a file **My_mesh.geo** with Gmsh, and you want to run a simulation on that mesh, you must first of all put the file inside the **geo/** folder. To tell the code that you want to run the simulation on that mesh, you have to set the MSH parameter to the name of the **.geo** file:

```
opts.MSH = "My_mesh";
```

Note that you must have at the end of the **.geo** file a line that saves the mesh in the MATLAB format. So, considering the previous example, at the end of the file **My_mesh.geo** you must have the line:

```
Save "My_mesh.m";
```

6.2 MSH_PARAMETERS

This parameter is a struct that contains in its fields extra inputs that you might want to pass to a parametric mesh. It is indeed possible to create **.geo** files that accept parameters from command line. For example you can create a mesh file that generates a rectangle, taking the width **W** and height **H** from command line. Supposing you want to pass **W** = 2 and **H** = 1 to the mesh file (that you have specified with the MSH parameter), you can do it with this syntax:

```
opts.MSH_PARAMETERS.W = 2;
opts.MSH_PARAMETERS.H = 1;
```

6.3 EPSR_VAL

This parameter is used to set the value of the relative dielectric permittivity ε_r in each region of the simulation domain. EPSR_VAL must be a **column** vector with as many elements as there are SurfaceIDs (see Section 4). Referring to the example in 4, if you wish to set ε_r to 1 and 4 in the 1 and 2 regions, respectively, you should set

```
opts.EPSR_VAL = [1; 4];
```

6.4 BCEL_FLAG

This parameter specifies the type of boundary conditions applied to the electric potential. BCEL_FLAG must be a **column** vector containing as many elements as there are BoundaryIDs (see Section 4). Each element of the vector can be set to 0, corresponding to a Dirichlet boundary condition at the associated external boundary, or 1, corresponding to a zero Neumann condition. Referring to the example in 4, if you want to apply a Dirichlet boundary condition at external boundaries 1 and 2, and a zero Neumann condition at external boundary 3, you should set

```
opts.BCEL_FLAG = [0; 0; 1];
```

6.5 BCEL_VAL

This parameters is used to set the value of the electric potential at those boundaries where a Dirichlet condition is used. BCEL_VAL must be a **column** vector containing as many elements as there are BoundaryIDs (see Section 4). Each element corresponding to a boundary with a Dirichlet condition should contain the value of the electric potential imposed at that boundary **normalized by** V_APPLIED. All the other elements of the vector (corresponding to zero Neumann boundaries) should be set to 0. Referring to the example in 4, if you want to apply an electric potential equal to half V_APPLIED at the boundary 1 and set the potential at boundary 2 to 0, you should set

```
opts.BCEL_VAL = [0.5; 0; 0];
```

6.6 V_APPLIED

This parameter is a function handle used to specify the applied voltage between the electrodes, as a function of time. The user can define any custom function.

```
opts.V_APPLIED = @(t)100*cos(2*pi*50*t + pi/4);
opts.V_APPLIED = @(t)20*(t-1e-9);
opts.V_APPLIED = @(t)20e3;
```

For DC discharges, it was observed that applying from the start the steady-state voltage may lead to failure of the time-integration solver. Gradually increasing the voltage until reaching the desired value often solved the problem. For this reason, some functions that implement this gradual increase of the voltage are provided in Merlino2D:

- `LinRamp`
- `LogRamp`
- `StairRamp` (particularly useful for obtaining the I-V characteristic of a corona discharge)

6.7 BC_FLAG

This parameter is used to specify the type of boundary condition for the species conservation equation. `BC_FLAG` must be a cell array with two columns.

In the first column, you should put strings containing the names of the species considered for the simulation. If `CHEMICAL_MODEL` was set to a **.m** script, the species must be the ones appearing in the `reactions` defined inside the script. If `CHEMICAL_MODEL` was set to `"Off"`, the species must be the ones specified in `SPECIES_NO_CHEM`. Note that all the species used must be defined inside the file **species_database.csv** contained in the **data/** folder.

Each element of the second column must be a row cell array with as many elements as there are BoundaryIDs (see Section 4). Each one of these cell arrays will be used to set the boundary condition for the continuity equation of the species that appears at the corresponding position in the first column of `BC_FLAG`. The *ith* element of one of these cell arrays must be a string specifying the type of boundary condition to be applied at the *ith* boundary. The available types of boundary conditions are (see the paper for details):

- `"Dirichlet"`
- `"Flux"`
- `"FreeDriftFlow"`
- `"GorinLike"`

For instance, considering a mesh with 3 BoundaryIDs and 2 species (`"e"` and `"N2+"`, that must be defined inside **species_database.csv**), you could set

```
opts.BC_FLAG = {
        "e", {"GorinLike", "Flux", "Dirichlet"};
        "N2+", {"GorinLike", "FreeDriftFlow", "Flux"}};
```

6.8 BC_VAL

This parameter is used to specify the value of boundary conditions for the species continuity equation. `BC_VAL` must be a cell array with two columns.

In the first column, you should put strings containing the names of the species considered for the simulation. If `CHEMICAL_MODEL` was set to a **.m** script, the species must be the ones appearing in the `reactions` defined inside the script. If `CHEMICAL_MODEL` was set to `"Off"`, the species must be the ones specified in `SPECIES_NO_CHEM`. Note that all the species used must be defined inside the file **species_database.csv** contained in the **data/** folder.

Each element of the second column must be a row cell array with as many elements as there are BoundaryIDs (see Section 4). Each one of these cell arrays will be used to set the boundary condition value for the continuity equation of the species that appears at the corresponding position in the first column of `BC_VAL`. The *ith* element of these cell arrays can be an anonymous function (if you wish to set a time-dependent value) or a double (if you wish to set a constant value), specifying the boundary condition to be applied at the *ith* boundary. Note that `BC_VAL` is complementary to `BC_FLAG`, and only boundary conditions of type `"Dirichlet"` or `"Flux"` require a value. Therefore, `BC_VAL` contains the desired value of number density (in m^{-3}) if a `"Dirichlet"` condition was selected, and the desired value of flux density (in $\mathrm{m}^{-2}\,\mathrm{s}^{-1}$) if a `"Flux"` condition was selected. For boundaries that were associated to `"GorinLike"` or `"FreeDriftFlow"` conditions, the corresponding value specified in `BC_VAL` will be ignored (but you must place some value anyway).

For example, considering the case of a mesh with 3 BoundaryIDs and 2 species, where you previously set

```
opts.BC_FLAG = {
        "e", {"GorinLike", "Flux", "Dirichlet"};
        "N2+", {"GorinLike", "FreeDriftFlow", "Flux"}};
```

you could set

```
opts.BC_VAL = {
        "N2+", {NaN, NaN, @(t)t^2};
        "e", {NaN, @(t)1e10*t, 1e15}};
```

With this combination of parameters you are imposing a constant number density of $1 \times 10^5\,\mathrm{m}^{-3}$ for the species `"e"` at the third boundary, and a time-dependent expression for computing the flux of `"e"` at the second boundary and of `"N2+"` at the third boundary. Notice that `NaN` have been placed in `BC_VAL` at positions corresponding to `"GorinLike"` and `"FreeDriftFlow"` flags. This was done to highlight that, for those species and boundaries where a `"GorinLike"` or a `"FreeDriftFlux"` condition has been used, the value inserted in the corresponding position of `BC_VAL` will be completely ignored.

6.9 TIME_INSTANTS

This parameter is a vector containing the time instants at which the solution is desired. The first value of the vector corresponds to the starting time of the simulation, and the last value of the vector tells the solver when to stop integration. If a vector with only two elements is provided, the solver will output all the steps that have been internally performed. Be careful in this case, because this might result in a lot of output time steps, that can quickly fill the memory, especially when meshes with a large number of triangles are used.

```
opts.TIME_INSTANTS = linspace(0,1e-6,100);
opts.TIME_INSTANTS = [0, 1e-6];
```

6.10 INITIAL_CONDITION

This parameter is used to specify the initial condition for the species number density. There are three types of use for this parameter:

- Uniform number density
- Load from previous simulation
- Gaussian distribution (only when `CHEMICAL_MODEL` = `"Off"`)

Uniform number density If you wish to assign a uniform number density to each species across the entire domain, set `INITIAL_CONDITION` to a cell array with two columns.

In the first column, you should put strings containing the names of the species considered for the simulation. If `CHEMICAL_MODEL` was set to a **.m** script, the species must be the ones appearing in the `reactions` defined inside the script. If `CHEMICAL_MODEL` was set to `"Off"`, the species must be the ones specified in `SPECIES_NO_CHEM`. Note that all the species used must be defined inside the file **species_database.csv** contained in the **data/** folder.

In the second column, you should put the desired uniform number densities in m^{-3}. Each of these values will be associated to the corresponding species in the first column.

```
opts.INITIAL_CONDITION = {
        "e", 1e10;
        "N2+", 1e12;
        "O2+", 1e13;
        "N2", 1e25};
```

Load from previous simulation If you wish to start a simulation using the number density distribution obtained at the end of a previous run, set `INITIAL_CONDITION` to a string containing the absolute path of the saved results file (the **.mat** file generated by the `Save` command). For example:

```
opts.INITIAL_CONDITION = "C:/Users/JohnSmith/Merlino2D/Results/test.mat";
```

Alternatively, you can specify a relative path; however, this approach is less reliable, as it may fail depending on the working directory at the time Merlino2D is executed. For instance, if the simulation is launched from the main Merlino2D folder, the following command is also valid:

```
opts.INITIAL_CONDITION = "Results/test.mat";
```

For this usage, the mesh should be the same of the previous simulation, as well as the species.

Gaussian distribution This option is available only when `CHEMICAL_MODEL` is set to `"Off"`. In this case, if you wish to assign to each species a two-dimensional Gaussian distribution, set `INITIAL_CONDITION` to a struct with fields

- A
- B

- x0
- y0
- sigma_x
- sigma_y

each of them being a vector with as many elements as the number of species contained in `SPECIES_NO_CHEM`. This will generate for the **ith** species contained in `SPECIES_NO_CHEM` a number density distribution

$$n_i(x, y) = \mathbf{A}[i] \exp\Bigg(- \Bigg(\frac{(x - \mathbf{x0}[i])^2}{(\mathbf{sigma\_x}[i])^2} + \frac{(y - \mathbf{y0}[i])^2}{(\mathbf{sigma\_y}[i])^2} \Bigg) \Bigg) + \mathbf{B}[i].$$

where the value of the parameters A, B, x0, y0, sigma_x and sigma_y is taken from the corresponding **i** element of the array. Note that the order of the numbers inside these arrays is important. For example you could set

```
opts.INITIAL_CONDITION.A = [1e10, 1e11];
opts.INITIAL_CONDITION.B = [1e5, 0];
opts.INITIAL_CONDITION.x0 = [1e-2, 0];
opts.INITIAL_CONDITION.y0 = [1e-2, 1e-1];
opts.INITIAL_CONDITION.sigma_x = [1e-3, 1e-3];
opts.INITIAL_CONDITION.sigms_y = [1e-1, 1e-1];
```

6.11 MU

This parameter is used to set the mobilities of the species, in $\mathrm{m^2\,V^{-1}\,s^{-1}}$. `MU` must be a cell array with two columns.

In the first column, you should put strings containing the names of the species considered for the simulation. If `CHEMICAL_MODEL` was set to a **.m** script, the species must be the ones appearing in the `reactions` defined inside the script. If `CHEMICAL_MODEL` was set to `"Off"`, the species must be the ones specified in `SPECIES_NO_CHEM`. Note that all the species used must be defined inside the file **species_database.csv** contained in the **data/** folder.

The elements in the second column will be used to determine the mobility of the corresponding species in the first column. Each element of the second column can be a number (if you wish to set a constant and uniform mobility) or a string. You should use a string to write an expression that will be used to compute the mobility. In the string, you can use any valid Matlab syntax to combine the available standard variables **E**, **Te**, **T** and **Ngas**, which correspond to the electric field in Td, the electron temperature in eV, the gas temperature in K, and the gas density in $\mathrm{m^{-3}}$, respectively. Note that the first three are arrays with as many elements as domain points, while the last one is a scalar. For example, you can set

```
opts.MU = {
        "e", 0.001;
        "O2-","exp(E/120)*T^(-0.5)";
        "O4+", "Ngas*max(log(T),Te^(-0.2))"};
```

to have a constant mobility for the species `"e"` and a mobility that is computed as a function of **E**, **Te**, **T**, and **Ngas** for the species `"O2-"` and `"O4+"`.

Inside the string expressions, you can also use **.csv** files contained inside the **data/** folder to define new variables (called csv variables). For example, suppose that you have obtained a relation between the reduced mobility and the reduced electric field, and that you have stored that relation inside a file **my_reduced_Mobility.csv**, where the first column contains the values of the reduced electric field, and the second column contains the values of the reduced mobility. In this case, you can set

```
opts.MU = {
        "O2-","exp(E/120)*T^(-0.5)";
        "e", 0.001;
        "O4+", "my_reduced_Mobility(E)/Ngas"};
```

to use the relation contained inside the **.csv** file to compute the mobility of `"O4+"`. Note that csv variables can be combined with standard variables, as you can see from the fact that `"my_reduced_Mobility"` is divided by the gas density `"Ngas"`. It is also possible to combine different csv variables inside a string expression.

Also, note that you must specify the dependence of the csv variable from one of the four standard variables. For instance, in the above example, `"(E)"` was added after the name of the csv variable to indicate that in the first column of the **.csv** file are stored the values of reduced electric field. If you wish to define a csv variable that depends on the value of the electron temperature, you must add `"(Te)"` after the name of the variable, like

```
opts.MU = {
        "O2-","exp(E/120)*T^(-0.5)";
        "e", 0.001;
        "O4+", "my_mob_as_func_of_Te(Te)"};
```

In case you set a value for `LOKI_INPUT`, additional variables (called loki variables) will be available. These are: **Loki_D**, **Loki_mu**, **Loki_alpha**, **Loki_eta**. They represent the reduced electron diffusion coefficient, the reduced electron mobility, the reduced Townsned ionization coefficient, and the reduced attachment coefficient, respectively. These variables work in the same way of csv variables, but are available without the need to define any **.csv** file, since they are obtained from the LoKI-B simulation specified with `LOKI_INPUT`. In addition, these variables are all dependent on the reduced electric field, which means that they must always appear inside a string expression as `Loki_D(E)`, `Loki_mu(E)`, `Loki_alpha(E)`, and `Loki_eta(E)`. For example, you could set

```
opts.MU = {
        "e", "Loki_mu(E)/Ngas";
        "N2+", 1e-4;
        "O-", "exp(E/120)*T^(-0.5)"};
```

6.12 D

This parameter is used to set the diffusion coefficient of the species, in $m^2\,s^{-1}$. `D` must be a cell array with two columns.

In the first column, you should put strings containing the names of the species considered for the simulation. If `CHEMICAL_MODEL` was set to a **.m** script, the species must be the ones appearing in the `reactions` defined inside the script. If `CHEMICAL_MODEL` was set to `"Off"`, the species must be the ones specified in `SPECIES_NO_CHEM`. Note that all the species used must be defined inside the file **species_database.csv** contained in the **data/** folder.

The elements in the second column will be used to determine the diffusion coefficient of the corresponding species in the first column. Each element of the second column can be a number (if you wish to set a constant and uniform value for the diffusion coefficient) or a string (if you wish to define an expression that will be used to compute the value of the diffusion coefficient). The exact same rules explained for the definition of a `MU` string expression (standard variables, csv variables and loki variables) also apply for `D`.

The only difference is that, when using a string expression for `D`, you can use a fifth standard variable, **mu**, in addition to the four previously described (**E**, **Te**, **T**, **Ngas**). With this variable, you can access the mobility of the species used in the simulation, and use it in the definition of the diffusion coefficient expression. Note that if you want to use the mobility of the species `"e"`, `"O3-"`, and `"N2+"`, you should write in the string expression `"(mue)"`, `"(muO3-)"`, and `"(muN2+)"`, respectively. The general syntax for using the **mu** variable is:

`"(mu"` + name of the desired species + `")"`.

This standard variable is particularly useful when you want to use the Einstein relation to compute the diffusion coefficient. For example, you can set

```
opts.D = {
        "N2+", "(muN2+)*T/11600";
        "e", "(mue)*Te";
        "O-", "(muO-)*T/11600"};
```

to use the Einstein relation for all three species. Alternatively, if a value was specified for `LOKI_INPUT`, you could set

```
opts.D = {
        "N2+", "(muN2+)*T/11600";
        "e", "Loki_D(E)/Ngas";
        "O-", "(muO-)*T/11600"};
```

6.13 V_TH_COEFF

This parameter is used to specify if the thermal velocity needs to be considered or not in `"GorinLike"` boundary conditions. This parameter should be a cell array with two columns.

In the first column, you should put strings containing the names of the species considered for the simulation. If `CHEMICAL_MODEL` was set to a **.m** script, the species must be the ones appearing in the `reactions` defined inside the script. If `CHEMICAL_MODEL` was set to `"Off"`, the species must be the ones specified in `SPECIES_NO_CHEM`. Note that all the species used must be defined inside the file **species_database.csv** contained in the **data/** folder.

Each element of the second column will be associated to the corresponding species in the first column and should be either a 0 (if the user wants to neglect the thermal velocity contribution) or a 1 (if the user wants to consider also the thermal velocity contribution). For example, in a simulation with 6 species, setting

```
opts.V_TH_COEFF = {
        "O3-", 0;
        "e", 0;
        "O4+", 0;
        "N2+", 1;
        "O2+", 1;
```

```
        "O-", 1};
```

means that for "O3-", "e" and "O4+" the thermal velocity contribution to the boundary flux density will be neglected, while it will be considered for "N2+", "O2+" and "O-".

6.14 CONST_OMEGA

This parameter is used to set a constant source term for the species. CONST_OMEGA must be a cell array with two columns.

In the first column, you should put strings containing the names of the species considered for the simulation. If CHEMICAL_MODEL was set to a **.m** script, the species must be the ones appearing in the `reactions` defined inside the script. If CHEMICAL_MODEL was set to "Off", the species must be the ones specified in SPECIES_NO_CHEM. Note that all the species used must be defined inside the file **species_database.csv** contained in the **data/** folder.

Each element of the second column should contain the constant and uniform value (in $\mathrm{m}^{-3}\,\mathrm{s}^{-1}$) of the source term to add to the $\partial n/\partial t$ of the corresponding species in the first column. This parameter can be useful to mimic background ionization and also to prevent the number density of species to decrease excessively.

```
opts.CONST_OMEGA = {
        "N2+", 0.8e15;
        "O2+", 0.2e15;
        "e", 1e15;
        "O2-", 1e5};
```

6.15 PHOTOIONIZATION

This parameter is a struct with fields: REACTIONS, SPECIES_COEFF, BC, and UPDATE_FREQUENCY.

REACTIONS is a cell array of strings. It is used to indicate which chemical reactions (contained in the kinetic scheme specified with CHEMICAL_MODEL) contribute to the ionization rate of neutral molecules by electron impact. The total ionization rate, S_i, will be computed as the sum of the reaction rates of the reactions specified. For example, you could set

```
opts.PHOTOIONIZATION.REACTIONS = {
        'e + N2 -> 2e + N2+';
        "e + O2 -> 2e + O2+"};
```

SPECIES_COEFF is used to distribute the total photoionization rate, S_{ph}, among different species. SPECIES_COEFF is a cell array with two columns. The first column contains the names of the positive ions produced (as strings) and the second column contains the weight associated to each positive ion.

Considering the previous example of PHOTOIONIZATION.REACTIONS, you may want to split S_{ph} between ${N_2}^+$ and ${O_2}^+$ with different weights. You could set

```
opts.PHOTOIONIZATION.SPECIES_COEFF = {
        "N2+",0.8;
        "O2+",0.2};
```

to add $0.8\,S_{ph}$ to the production rate of ${N_2}^+$ and to add $0.2\,S_{ph}$ to the production rate of ${O_2}^+$. S_{ph} will be automatically added to the production rate of electrons.

BC is used to set the boundary conditions for the Helmholtz equation

$$\nabla^2 S_{ph}^{(j)} - (\lambda_j p_{O_2})^2 S_{ph}^{(j)} = -A_j p_{O_2}^2 I.$$

BC must be a vector containing as many elements as there are BoundaryIDs (see Section 4). Each element of the vector can be set to 0, corresponding to a **zero** Dirichlet boundary condition at the associated external boundary, or 1, corresponding to a **zero** Neumann condition. For example, you could set

```
opts.PHOTOIONIZATION.BC = [1, 1, 0, 1];
```

UPDATE_FREQUENCY is an integer used to specify the frequency at which the photoionization rate S_{ph} is updated.
Setting

```
opts.PHOTOIONIZATION.UPDATE_FREQUENCY = 1;
```

means that the rate S_{ph} will be recomputed at each time instant contained inside TIME_INSTANTS.
Setting

```
opts.PHOTOIONIZATION.UPDATE_FREQUENCY = 5;
```

means that the rate S_{ph} will be recomputed once every 5 time instants.

6.16 CHEMICAL_MODEL

This parameter is a string used to specify the kinetic scheme to use in the simulation. The default value for this parameter is

```
opts.CHEMICAL_MODEL = "Off";
```

which means that no kinetic scheme will be used. This is equivalent to solving the drift-diffusion equations without considering the source term Ω. If you want to use a kinetic scheme, you must first of all create a **.m** script and put it inside the **kinetic/** folder. In this script, you should define the details of the kinetic scheme you wish to use. You can then assign to CHEMICAL_MODEL the name of the file you created. For example, if you created the script **my_kinetic_scheme.m**, you can set

```
opts.CHEMICAL_MODEL = "my_kinetic_scheme";
```

to use the reaction set defined inside that file. Note that the extension **.m** must be omitted.

A kinetic scheme script must contain the variable `reactions`, that is a cell array, with two columns and as many rows as the number of chemical reactions included in the kinetic scheme. The first column should contain the definition of the reactions, the second should contain the definition of the corresponding rate coefficients (in $\mathrm{m^3\,s^{-1}}$, if there are two reactants, or $\mathrm{m^6\,s^{-1}}$, if there are three reactants). Each element of the first column should be a string containing the definition of a chemical reaction, with the syntax:

```
"A + B -> C + D"
"2A + E -> C+ + B"
```

Each element of the second column can be a number (if you wish to set a constant and uniform value for the rate coefficient) or a string (if you wish to define an expression that will be used to compute the rate coefficient). In the second case, the exact same rules explained for the definition of a MU string expression (standard variables, csv variables, and loki variables) also apply.

For the definition of rate coefficients of reactions involving electrons, in case a value has been specified for LOKI_INPUT, it is also possible to use the rate coefficient computed with

LoKI-B. This can be done by setting the element of the second column to a specific string. LoKI-B will compute the rate coefficient for all the reactions taken as input (contained inside LXCat files). Therefore, to use a rate coefficient computed by LoKI-B, first of all it is necessary to identify the desired reaction. For example, you might want to use, as rate coefficients of one of the reactions you defined in the first column of `reactions`, the value that LoKI-B calculated for the following reaction (contained inside the LXCat files):

```
IONIZATION
O2 -> O2+
 1.210000e+1
SPECIES: e / O2
PROCESS: E + O2 -> E + E + O2+, Ionization
PARAM.:  E = 12.1 eV, complete set
COMMENT: [e + O2(X) -> e + e + O2(+,X), Ionization]
COMMENT: Information Center Report, University of Colorado
UPDATED: 2017-12-31 18:08:01
COLUMNS: Energy (eV) | Cross section (m2)
-----------------------------
 1.210000e+1    0.000000e+0
 1.220000e+1    6.000000e-23
 1.230000e+1    7.000000e-23
 1.240000e+1    8.000000e-23
```

To achieve this, you need to set the corresponding element of the second column of `reactions` to a specific string, that will allow Merlino0D to select the right rate computed by LoKI-B. This string is the one **inside the square brackets** after the text "`COMMENT:`" in the LXCat files. For this specific example, the string to put in the second column of `reactions` is

`"e + O2(X) -> e + e + O2(+,X), Ionization"`

In Merlino2D some kinetic schemes for air are already provided inside the folder **kinetic/** and can be used as reference for the creation of new scripts, also for other types of gases.

6.17 CONST_SPECIES

This parameter is a cell array with 3 columns, used to specify the species that need to be considered constant and uniform during the simulation (if there are any) and their respective value.

The first column should contain the name of the species (as strings) to treat as constant. All the species should appear inside the kinetic scheme specified with the `CHEMICAL_MODEL` parameter.

The second column contains the constant value of the associated species, that can be an absolute value or a relative value with respect to the gas density, depending on the content of the third column. If the third column contains the string `"abs"`, the number in the corresponding second column will be interpreted as a constant number density (in m^{-3}). If the third column contains the string `"rel"`, the number in the corresponding second column will be interpreted as a fraction of the gas density `Ngas`.

For example, you could impose a constant number density for the species `"A"` equal to 80% of the total gas density, and a constant number density for the species `"B"` equal to $1 \times 10^{15}\,\mathrm{m}^{-3}$ by setting

```
opts.CONST_SPECIES = {
        "A", 0.8, "rel";
        "B", 1e15, "abs"};
```

Note that all the species defined in `CONST_SPECIES` must be removed from the input parameters `BC_FLAG`, `BC_VAL`, `INITIAL_CONDITION`, `MU`, `D`, `V_TH_COEFF`, and `CONST_OMEGA`, since they will not be part of the degrees of freedom of the simulation.

If no species need to be considered as constant, you can avoid to define `CONST_SPECIES`.

6.18 LOKI_INPUT

This parameter is a string used to specify the **.in** input file for LoKI-B. To run a new LoKI-B input file, you must first of all create a valid one (we refer the reader to LoKI-B documentation), and place it inside the **Input/** folder of the LoKI-B directory (note that it is not possible to put the file inside a sub-folder). You can then set `LOKI_INPUT` to the name of the input file, removing from the name the **.in** extension.

For example, if you created a LoKI-B input file named **my_loki_input.in** and you put it inside the **Input/** directory (already present in LoKI-B distribution), you should set

```
opts.LOKI_INPUT = "my_loki_input";
```

To avoid running again the same file in the future, and load directly the results, it is suggested to save the results directly from the LoKI-B input file, by putting these settings at the end:

```
% ---  configuration of the output files ---
output:
  isOn: true
  dataFormat: hdf5
  folder: my_loki_input
  dataSets:
    - eedf
    - swarmParameters
    - rateCoefficients
```

Note that the text `my_loki_input` after the `folder` field should be replaced with the actual name of the **.in** file (`my_loki_input` was used here to be consistent with the previous example). If the shown settings are added to the **.in** file, the next time a simulation is run with the option

```
opts.LOKI_INPUT = "my_loki_input";
```

the results will be loaded directly without the need to run again LoKI-B.

The input file for LoKI-B, **Air.in**, for a mixture of N_2(79 %) - O_2(21 %), is already provided with Merlino2D.

6.19 ELECTRON_TEMPERATURE

This parameter is used to set the electron temperature for the simulation. You can select a constant and uniform value of the electron temperature (in eV) for the simulation by setting `ELECTRON_TEMPERATURE` to that value

`opts.ELECTRON_TEMPERATURE = 2;` (default)

Alternatively, you can set the electron temperature to be a function of the electric field. In this case, you need to prepare a **.csv** file containing the relationship between the reduced electric field (in Td) and the electron temperature (in eV). The electric field values must be in the first column of the file, the electron temperature values in the second column. This file must be placed inside the folder **data/** in Merlino2D main directory. You can then set `ELECTRON_TEMPERATURE` to the name of that file

```
opts.ELECTRON_TEMPERATURE = "my_E_Te_relation";
```

The file **Te_Air.csv**, containing the electron temperature dependence on the reduced electric field for air, is already provided in Merlino2D and can be used as reference for creating new files.

In addition, in case a value is provided to `LOKI_INPUT`, you can set

```
opts.ELECTRON_TEMPERATURE = "LoKI";
```

to use the electron temperature (as a function of the reduced electric field) that was computed by LoKI-B.

6.20 TEMPERATURE

This parameter is the gas temperature in Kelvin. At the current stage, a constant and uniform value is considered. The default is

```
opts.TEMPERATURE = 300;
```

`TEMPERATURE` is used to compute the gas density in m^{-3}, `Ngas` (mentioned in the explanation of `MU`, `D`, and `CHEMICAL_MODEL`), with the equation:

$$N_{gas} = \frac{P}{k_B T}$$

where k_B is the Boltzmann constant, P is the pressure in Pa, and T is the temperature in K.

6.21 PRESSURE

This parameter is the gas pressure in Pa (Pascal). The default is atmospheric pressure

```
opts.PRESSURE = 101325;
```

`PRESSURE` is used to compute the gas density in m^{-3}, `Ngas` (mentioned in the explanation of `MU`, `D`, and `CHEMICAL_MODEL`), with the equation:

$$N_{gas} = \frac{P}{k_B T}$$

where k_B is the Boltzmann constant, P is the pressure in Pa, and T is the temperature in K.

6.22 ELECTRON_REF_COEFF

This parameters is the electron reflection coefficient that will be used in `"GorinLike"` boundary conditions. The default is

```
opts.ELECTRON_REF_COEFF = 0;
```

6.23 GAMMA_II

This parameter is the secondary electron emission coefficient that will be used in boundary conditions of type `"GorinLike"`. The default is

```
opts.GAMMA_II = 1e-2;
```

6.24 SURF_CHARGE_COEFF

This parameter is a vector with three elements. It is used to set the dielectric interfaces accumulation coefficients K, which can vary between 0 and 1. The vector contains three elements, independently from the total number of species in the simulation. The first element will be the accumulation coefficient for the electrons, the second element will be the accumulation coefficient for all positive ions, and the third element will be the accumulation coefficient for all negative ions. For example, by setting

```
opts.SURF_CHARGE_COEFF = [0, 0.1, 1];
```

at dielectric interfaces no electrons will deposit their charge, only 10 % of positive ions will deposit their charge, and the totality of negative ions will deposit their charge. The default for this parameter is

```
opts.SURF_CHARGE_COEFF = [1, 1, 1];
```

6.25 GAMMA_II_DIEL

This parameter is the secondary electron emission coefficient that will be used at dielectric interfaces. The default is

```
opts.GAMMA_II_DIEL = 1e-2;
```

6.26 ODE_TYPE

This parameter is a string for selecting the solver used for time integration of the DAE system. The available options are:

- `opts.ODE_TYPE = "ode15s";` (default)
- `opts.ODE_TYPE = "idas";`

When selecting `"ode15s"`, the MATLAB-native *ode15s* routine is called to integrate the system. When selecting `"idas"`, the *idas* solver from Sundials is called instead. The latter option has some disadvantage: `OUTPUT_FUNCTION` will default to `"none"` and no statistics from the solver will be available at the end of the simulation. On the other hand, it offers a significative reduction of the computational time, up to 50% in some cases.

6.27 OPEN_GMSH

This parameter is a flag that allows the user to select if the computational mesh should appear before the simulation starts. By default, this flag is set to 1,

```
opts.OPEN_GMSH = 1;
```

which means that before every simulation starts, a window showing the mesh will appear. The user can inspect the mesh, and **only after closing** this window, the simulation will effectively begin. In case multiple simulations have to be run in sequence, it is desirable to disable this function,

```
opts.OPEN_GMSH = 0;
```

so that the simulations run one after the other, without the user needing to close the Gmsh window before the next simulation starts.

6.28 REORDERING

This parameter is a flag that, when set to 1,

```
opts.REORDERING = 1;
```

allows the user to perform symmetric reverse Cuthill-McKee ordering of the jacobian matrix. This resulted in a speed-up of the computation in some cases. However, the reduction of the computational time was not particularly relevant in the tests performed so far. For this reason, the default for this parameter is

```
opts.REORDERING = 0;
```

Setting this flag to 1 may result in greater improvement of the performance when considering larger meshes or a big number of chemical species.

6.29 OUTPUT_FUNCTION

This parameter is a string that allows the user to specify the output function. The available options are:

- `opts.OUTPUT_FUNCTION = "bar";` (default)
- `opts.OUTPUT_FUNCTION = "current";`
- `opts.OUTPUT_FUNCTION = "none";`

When selecting `"bar"`, a progress bar will appear, allowing the user to monitor the progress of the simulation. The option `"current"` is for automatically stopping the simulation at steady-state in a DC discharge. When this option is selected, a plot of the current (computed integrating the current density over the external boundary that has identifier 1, see Section 4) in time will appear during the progress of the simulation. The parameters `STEADY_STATE_THRESHOLD` and `T_START_STEADY_STATE` are used to determine when the steady-state is reached. No output function is used when `"none"` is selected.

6.30 BAR_SCALE

This parameter is used when `"bar"` is selected as `OUTPUT_FUNCTION`. The available options are:

- `opts.BAR_SCALE = "lin";` (default)
- `opts.BAR_SCALE = "log";`

When selecting `"lin"`, the scale on the progress bar will be linear, when selecting `"log"` the scale on the progress bar will be logarithmic. The latter can be particularly useful when simulating a DC discharge trying to obtain the steady-state solution. When using the `"log"` scale, the first time instant of the simulation should not be zero.

6.31 STEADY_STATE_THRESHOLD

This parameter is the tolerance at which the steady-state condition is considered to be reached.

`opts.STEADY_STATE_THRESHOLD = 1e-3;` (default)

It is used when `"current"` is selected as `OUTPUT_FUNCTION`. When the relative difference (compared to the previous time step) of all the unknowns is less than the value specified in `STEADY_STATE_THRESHOLD`, Merlino2D stops the simulation, assuming steady-state was reached.

6.32 T_START_STEADY_STATE

This parameter is the time at which the code starts looking for steady-state, using the value specified in `STEADY_STATE_THRESHOLD` as tolerance. It is used when `"current"` is selected as `OUTPUT_FUNCTION`. If you are running a DC discharge simulation, and you are reasonably sure that the steady-state will not be reached before 1 µs, it may be a good idea to let the code know that it should start checking for a steady-state condition only after this time.

```
opts.T_START_STEADY_STATE = 1e-6;
```

The default value for this parameter is

```
opts.T_START_STEADY_STATE = 0;
```

6.33 ABS_TOL

This parameter is the absolute tolerance for the DAE solver. The default is

```
opts.ABS_TOL = 1e-6;
```

6.34 REL_TOL

This parameter is the relative tolerance for the DAE solver. The default is

```
opts.REL_TOL = 1e-3;
```

6.35 SPECIES_NO_CHEM

This parameter is used when `CHEMICAL_MODEL` is set to `"Off"`. `SPECIES_NO_CHEM` must be a cell array containing the strings that identify the species that the user wants to consider for the simulation. The order in which the species appear is important in case a Gaussian distribution is used in `INITIAL_CONDITION`. Note that all the species used must be defined inside the file **species_database.csv** contained in the **data/** folder.

```
opts.SPECIES_NO_CHEM = {"N2+", "O2-", "O-", "N", "N2";}
```